

TourPay invoicing: what Accufy has, what we have, what to add

Studied 2026-09-24: Accufy v3.0 (~/Downloads/accufy-saas-financial-software/accufy_v3.0 , a Codelgniter accounting product, 5,277 files, 6 public controllers plus 36 admin controllers) against the portal's two invoice screens.

1. The first finding: there are two invoice modules, and I built on the wrong one

	TourPay (<code>modules/TourPay</code>)	Vendor "Invoices" (<code>modules/Vendor</code> , Finance menu)
Table	<code>bc_tourpay_invoices</code> + items	<code>bc_vendor_invoices</code> + lines + payments
Screens	List, 4-step form, view, 5 PDF templates, public pay page, e-mail, WhatsApp, share link	One small form and a list
Types	Invoice and quotation	Invoice only
Owner column	<code>author_id</code> (no tenant scope)	<code>vendor_id</code> (tenant scope)
Payments	Only "mark paid" (all or nothing)	Ledger with partial payments, void, from-booking
Public page	Yes, <code>/tourpay/pay/{token}</code> , but "Pay Now" is a placeholder alert	None
API and webhooks	None	All of Phase 0 and the API work (12 endpoints, 3 events)
Production data	2 invoices	1 invoice

You said the invoice section is meant to be **TourPay**. Agreed: TourPay has the client-facing side (templates, quotations, share link, PDF) and the Vendor screen is the primitive duplicate that I extended without checking. That breaks the "extend, don't duplicate" rule. The API, webhooks and payments ledger I built need to move onto TourPay, and the Vendor "Invoices" screen should go.

Problems found in TourPay itself (all fixable):

- **"Pay Now" does nothing.** It shows an alert. The portal already has PayPal, Stripe, Paystack, Payrex and 2Checkout gateways, and `BookingPayments` .
- **Marking an invoice paid completes the linked booking.** Paid is not the same as trip done, and completing awards loyalty points. It must be a payment, not a status jump.

- **No partial payments, deposits or balance due**, though tours are sold on deposit and balance.
- **Not tenant scoped**. It filters on `author_id` by hand; no `BelongsToVendor`, no isolation test.
- **Invoice numbers are global**, counted across all tenants, with a prefix taken from the site title, not the vendor.
- **E-mail failures are swallowed**: a failed send is silent and the invoice may not be marked sent.
- One tax rate, default currency ZAR, no discount, no customer (CRM) link, no booking link except a Tanova trip.

2. What Accufy has (and what matters for a tour business)

Accufy's invoicing features, from its controllers, views and schema:

Accufy feature	Detail	For tours
Invoice, estimate, credit note, bill	One table, <code>type</code> 1 to 4 plus recurring flag	Yes : quotation, invoice, credit note. Bills become supplier bills.
Status flow	Draft, unpaid, partial, paid, plus sent and viewed tracking (no stored overdue state)	Yes , and we add overdue
Partial payments with ledger	<code>payment_records</code> per invoice (date, method, note, proof), status follows the total	Yes
Customer advance balance	Overpayment is kept as credit and offered on the next invoice	Yes (deposits and overpayments)
Online payment on the customer page	Stripe, PayPal, Razorpay, Paystack	Yes (we have five gateways already)
Offline payment with proof upload	Customer uploads a bank slip, owner approves	Yes , EFT is common here
Public customer page	View, print, PDF, pay, approve or reject an estimate, "viewed" tracking	Yes
Estimate to invoice	Convert with one click	Yes , quotation to deposit invoice
Multi-currency	Customer currency, live rates, converted total	Yes , international guests
Taxes	Named tax types, several per invoice, recoverable flag, per-product tax	Yes, several taxes and tourism levy
Discount	Per invoice	Yes
Products/services catalogue	Pick items, save new ones, units, stock	Use our listings, add-ons and tiers instead

Accufy feature	Detail	For tours
Invoice numbering	Automatic prefix and counter per business	Yes , per vendor
Templates	8 styles, colour, logo, titles, footers, QR code	We have 5 styles; add colour, logo, footer, QR
Send	E-mail with message, copy to self, SMS text, receipt e-mail	Yes , plus WhatsApp (we have it)
Recurring invoices	Frequency, start and end, auto-send by cron	Instalment plans, not subscriptions
Clone, share link, print, PDF download		Yes
Reports	Profit and loss, sales tax, customer and vendor statements, balance sheet, invoice details	Yes : receivables, tax, statements, revenue by service
Expenses and bills	Expense entries, vendor bills with payments	Supplier bills tie to our Operators
Roles and permissions, multi-business		Portal already has team roles
Not relevant	HRM (employees, salary, attendance), blog, testimonials, referral, SaaS plans and packages	Skip

Notes on Accufy: it is a commercial CodeCanyon product, so we take **ideas only, no code**. Its `Cron.php` contains a hard-coded currency-API key, and it stores money in loose columns with float comparisons (`$amount == $invoice->grand_total`); we should not copy either.

3. What to add to TourPay, in tour-business terms

A. Make it correct (foundation)

1. Tenant scope (`vendor_id` plus `BelongsToVendor`), per-vendor invoice numbers with a configurable prefix and next number, and isolation tests.
2. A payments ledger on TourPay (date, amount, method, reference, proof, who recorded it). Status follows the money: draft, sent, viewed, partly paid, paid, overdue, void. Overdue is calculated from the due date, not a stored guess.
3. Marking paid records a payment. It never completes the booking; completing stays a separate, deliberate step.
4. Failed e-mail and WhatsApp sends are shown and can be retried.

B. Get paid (the missing half)

1. **Working Pay Now** through the existing gateways (PayPal, Stripe, Paystack, Payrexx), for the invoice balance, with a receipt e-mail.
2. **Deposit and balance**: an invoice can carry a schedule (30% now, balance 14 days before travel), reusing `BookingPayments::buildPlan`; each instalment has its own due date and pay link.
3. **EFT with proof**: the guest uploads the bank slip on the pay page; the vendor approves it in one click and the payment is recorded.
4. **Reminders**: before due, on due, and every N days overdue, by e-mail or WhatsApp, with a per-vendor switch (this is what the scheduled-messages engine is for).
5. Overpayment kept as a credit on the guest's account.

C. Sell better

1. **Quotation lifecycle**: sent, viewed, accepted or declined by the guest on the public page, expires after `valid_days`; accepting creates the deposit invoice.
2. **Link to what was sold**: booking, customer (CRM), departure and add-ons, so lines can be pulled from a booking, a tier and its add-ons, or a Tanova trip. Group and per-person pricing as line types.
3. **Multi-currency**: invoice in the guest's currency with the rate and the base-currency total stored on the invoice.
4. Several named taxes per invoice, discount (amount or percent), service fee, and a tourism levy line.
5. **Credit notes and refunds**: issue against an invoice, feeding the existing refund record.
6. Templates: keep the 5, add logo, brand colour, footer and terms per vendor, and a QR code to the pay link.
7. "Viewed" tracking and a proper timeline (created, sent, viewed, paid, reminded), reusing the booking timeline component.

D. See the money

1. **Receivables**: who owes what and how late (0 to 30, 31 to 60, over 60 days).
2. Statements per customer, tax report, revenue by service and by month, exportable as CSV and PDF.
3. **Supplier bills**: record what you owe operators (uses the Suppliers module and its fares) with payments, and a simple profit per booking (sold minus supplier cost).

E. API and integrations (rebuilt on TourPay)

1. The `/invoices` endpoints, their payments and lines, the three invoice webhooks, the "invoice from booking" action and the documentation move to TourPay tables **with the same response shapes**, so nothing already documented breaks. New endpoints for quotations, credit notes, reminders, receivables and

statements.

4. How to do it without duplicating

1. **Do not build a third module.** Extend TourPay's tables and controller.
2. **Migration:** add `vendor_id` (fill from `author_id`), `customer_id`, `booking_id`, `parent_id`, `paid_amount`, `discount`, `exchange_rate`, `number_seq` and new `type` values to `bc_tourpay_invoices`; add `bc_tourpay_payments`, `bc_tourpay_taxes`. Move the single Vendor invoice across. Production has 3 invoices in all, so this is cheap now and expensive later.
3. **Reuse:** `BookingPayments` and the gateways for pay and refunds, `WaitlistNotifier` -style channel dispatch for reminders, `InvoiceBook` and `InvoiceFromBooking` logic (moved, not rewritten), `ListQuery` and `FilterBar` for the list, the webhook events for invoice changes.
4. **Retire** the Vendor "Invoices" screen: redirect `/vendor/invoices` to TourPay, remove the Finance menu duplicate, keep the API paths.

5. Order of work

Phase	Content	Size
T1	Foundation A (scope, numbers, ledger, correct paid, tests) and merge of the two modules, API re-pointed	3 to 4 days
T2	Get paid: working Pay Now, deposit and balance, EFT with proof, receipts	3 to 4 days
T3	Quotation lifecycle, booking and CRM links, multi-currency, taxes and discount, credit notes	3 to 4 days
T4	Reminders, receivables, statements and reports, supplier bills	3 days
T5	Templates and polish (logo, colour, QR, timeline), API additions and docs	2 days

Payments in T2 need PayPal credentials on production (not configured today); Stripe or Paystack work if you have those instead.

6. Questions for you

1. Confirm: TourPay becomes the only invoicing module and the Vendor "Invoices" screen goes.

2. Which gateway do your vendors use to be paid online: PayPal, Stripe, Paystack, or bank transfer only? Money should go to each vendor's own account, not the platform's; today the portal's gateway keys are global.
3. Invoice currency: one currency per vendor, or per guest?
4. Do you want supplier bills and profit per booking now, or later?
5. Should reminders be on by default for new vendors, or opt in?

7. Status (2026-09-24, built and tested locally; not deployed)

Your answers: TourPay is the only invoicing module; each vendor sets their own gateways in settings (their account, not the platform's); one currency per invoice and a business can bill in several; supplier bills and profit per booking now; reminders are set per vendor in settings.

Phase	State	What is there
T1 Foundation and merge	Done	One module. Tenant scope, per-vendor numbers (<code>INV-2026-001</code> , prefix set by the vendor), payments ledger with partial payments, status follows the money, "mark paid" records the balance and never completes a booking, failed sends are reported, per-vendor settings. The Vendor "Invoices" screen, routes and menu are removed (old links redirect); the invoice API, webhooks and "invoice from booking" now run on TourPay with the same response shapes. Existing invoices copy across in the migration.
T2 Get paid	Done	Working Pay Now through the vendor's own Stripe, PayPal, Paystack, Paynow (Zimbabwe) , Pesapal and Selcom (Tanzania) (keys encrypted, never shown again, a "test connection" button per gateway; a gateway only appears on an invoice whose currency it can take). Providers that call us back (Pesapal, Selcom, Paynow) hit one public notify address that only ever asks the provider about our own attempts; the message itself is never believed. Recorded only when the gateway confirms, once; a background check finds payments the guest never came back for. Bank transfer: the guest reports it with a reference and proof, it waits until the vendor confirms. Deposit and balance or equal instalments; the guest can pay just the next one. Receipts by e-mail. WhatsApp uses the vendor's own connected number, not a platform key.
T3 Sell better	Done	Quotation lifecycle (guest accepts or declines on their link, expires, converts to one invoice). Several named taxes, tax-inclusive or exclusive, discount. Credit notes (with their share of tax) and refunds. Booking and customer links, "add from your tours and add-ons" on the form, base currency with the vendor's own rates for totals across currencies.
T4 See the money	Done	Reminders (off until a vendor turns them on; before due, then every few days overdue, capped, never twice a day; e-mail or their WhatsApp). Reports: who owes you by age, money in by month, tax, client statements, profit by booking; all as CSV. Supplier bills with payments, and a profit card on every booking .
T5 API and docs	Done	44 new endpoints (invoices: credit note, refund, schedule, convert, duplicate, send, confirm transfer, settings, reports; bills; booking profit), all documented, contract-tested, plus a new *Invoicing, payments and supplier bills* guide.

Accufy ideas used (ideas only, no code): status tabs with counts, "amount due" and days-late columns, one row menu of actions, record-a-payment modal pre-filled with the balance, a payment panel with a progress bar and a history, due-date shortcuts (on receipt, 7, 14, 30 days), template and defaults per business, a client-facing page with a preview mode and a viewed indicator.

Settings are a proper page now (`/user/tourpay/settings` , no pop-ups): Invoices, Getting paid (bank details, each gateway with help text, currencies and a test button), Currencies and rates, Reminders, and an Activity trail (who recorded, confirmed, refunded, voided or changed settings, and when).

Tests: 323 pass (`tests/Unit` , `tests/Feature/Vendor` , `tests/Feature/Api`). Gateways are faked at the HTTP layer; **nothing here has been run against real Stripe, PayPal, Paystack, Paynow, Pesapal or Selcom accounts**. Paynow's status check and Selcom's request signing were written from the providers' documentation and are the least proven, so a vendor's first payment on any gateway should be a small one.

Platform hardening done alongside (2026-09-24, local, not deployed)

- Vendor API CORS: a wildcard in the framework config was overriding each business's registered origins; now only registered origins are answered (test added).
- Another business's team invitation could be re-sent by id; fixed, and cross-tenant tests added for keys, origins, webhooks and the concierge API.
- Audit trail table for money actions; `/health` (database, disk, scheduler heartbeat, 503 on failure) for an uptime monitor; `php artisan tenant:export {vendor_id}` writes one business's data (no passwords, tokens or keys) to a zip.
- Dependencies: 71 known security advisories down to 1 low (`firebase/php-jwt` , no fix available). Laravel 12.25 to 12.69, Symfony, Guzzle, CommonMark, PhpSpreadsheet and others. The deploy script now runs `composer install` .
- Backup script can copy each verified dump to S3 (`OFFSITE_S3_URI`) and has a `--restore-test` mode for a weekly restore check. CI workflow added (`.github/workflows/ci.yml` : syntax, route and view compile, `composer audit`); it starts working once the portal repository has a remote.

One ledger (2026-09-25, local, not deployed): the single source of truth for money

- **Before:** money lived in four places that only agreed if nothing went wrong: the booking's stored `paid` , the booking page's own ledger, TourPay's invoice payments, and the platform gateways writing `paid` directly. A payment on an invoice never reached the booking; the "invoice from booking" copied `paid` once and then drifted; the payout balance was worked out from booking totals, not money received; nothing was locked, so two payments at once could be lost or overpay.
- **Now:** one append-only table (`bc_money_ledger`) holds every confirmed movement of money: client payments and refunds, supplier payments, payouts. Each fact has a key and is written once. The database itself refuses to change or delete a row (triggers; the model refuses too). A mistake is fixed by a reversing row.

- **Everything else is derived:** a booking's `paid`, an invoice's paid amount, a bill's paid amount, the Finance statement, profit and the payout balance. The booking's stored `paid` is a cache the ledger keeps equal, even when a gateway writes a stale number.
- **Booking and invoice are one story:** a payment on either counts on both (same currency), refunds too; the booking status follows the money but never becomes "completed" because of a payment.
- **Payouts:** every row says who holds the money (`vendor` or `platform`). The payout balance counts only money the platform actually holds, capped at the vendor's share and lowered by refunds the vendor gave.
- **Safe under load:** the booking, invoice or bill row is locked while its balance is checked (always booking first, then invoice, with automatic retry on a deadlock); duplicates are recognised by key; the same payment twice within a minute on a booking is one payment.
- **Checked every night:** `php artisan money:reconcile` compares every stored total with the rows behind it and writes what it finds to `/health`, which turns red on any difference. `--fix` writes missing rows and refreshes stored totals; it never bends the ledger to fit. `php artisan money:backfill` loads pre-ledger money (it also runs once as a migration).
- **Rehearsed** against a restore of the production database: migration, backfill (no money recorded yet on production) and reconcile were clean.
- **Tests:** 350 pass, including invariants (every stored total equals its rows after a mixed run), idempotency, locking, the stale-gateway case and payouts.
- **Known limits closed (2026-09-25):**

- **Cross-currency invoices:** an invoice keeps its own currency and is never converted. When its booking is in another currency, the booking's paid amount is credited at that day's rate: the business's own rates first, otherwise the free daily feed at `open.er-api.com` (kept per day in `bc_fx_rates`, refreshed nightly by `fx:refresh`; attribution "Rates By Exchange Rate API" is shown on the statement). With no rate known nothing is guessed: the payment is not linked and the nightly check lists it. A booking payment in one currency is allocated to its invoice in the other the same way. - **Commission:** money a business collects itself (its own gateway, bank, cash) accrues the platform's commission share, in the currency it was paid in (currencies are never mixed). Fully paid owes exactly the booking's commission; a refund gives its share back. It shows on the statement as "Commission owed to platform" (an account, not cash). In the payout currency it is held back from the next payout and settled out of what the platform still holds; a debt in another currency is never set against money in this one.

`commission:settle {vendor} {currency} {amount}` records commission paid outside the platform. Money already in the ledger before commission existed gets its commission once, from the backfill; the nightly check flags any payment that missed it. - **One payment schedule:** when a booking has a live invoice in the same currency, its plan and the invoice's instalments are the same schedule. An edit on either screen (build, clear, waive, split, deposit) is copied to the other, paid rows follow the ledger earliest first, and a refund takes the last ones back to pending. - **Nightly check at scale:** every check is one set-based SQL query (no query per payment); it stays a handful of queries for thousands of payments (tested) and adds two checks: payments that could not be linked for want of a rate, and, in production, that the database triggers exist. - **Tamper-evident on any server:** every ledger row carries a hash of its own content and of the row before it (one chain per business), and an anchor remembers each chain's last row. A changed amount, a deleted row, a row slipped in, or newest rows removed all break the chain, and the nightly check turns `/health` red. This needs nothing from the server. Database triggers that also refuse the change outright are an optional

extra layer: on a server with binary logging they need `log_bin_trust_function_creators = 1`, then `php artisan money:protect`; `/health` reports which layers are active.

Before this goes live

1. The three TourPay migrations run on deploy (they copy the one Vendor invoice on production across, and change the invoice-number index to per-vendor).
2. Reminders and the payment check need the scheduler. Both are switched on in `routes/console.php` but only act for vendors who opted in (reminders) or have open payment attempts (check). The older never-run jobs stay off behind `SCHEDULE_LEGACY_TASKS`.
3. Each vendor must enter their own gateway keys before Pay Now appears on their invoices.
4. Deploy runs `composer install` (the lock file changed) and one new migration (`bc_audit_log`).